

需求分析方法与 AI 辅助建模

软件工程与计算 II · 模块二 · 单元 4

从用户的声音到精确的模型——需求分析让沟通变为共识

课程	软件工程与计算 II
专业	软件工程
工具	PlantUML · Mermaid · LLM 辅助建模

今日学习路径

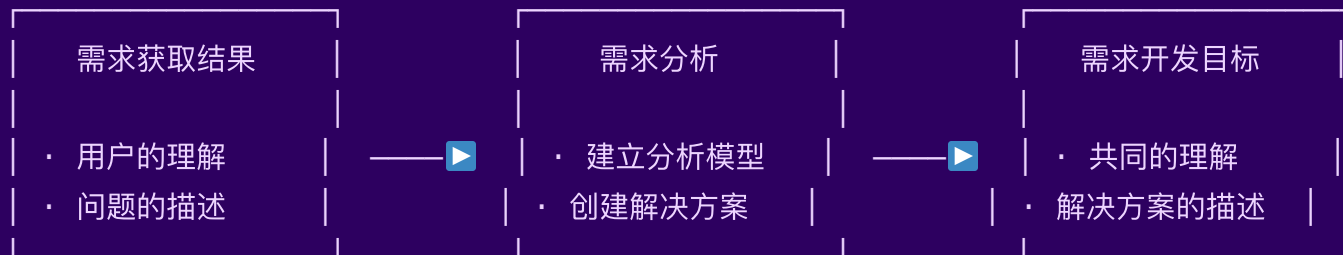
① 为什么分析	② 分析模型概览	③ OO分析核心图	④ AI生成UML	⑤ 实操与评测
从获取到理解 桥梁与目标	常见分析模型 OO分析过程	类图/顺序图/状态图 三大视角	LLM+PlantUML 质量评估	课堂练习·打分 对比与反思

三个核心问题，今天结束时你应该能回答：

1. 需求分析的目的是什么？有哪些**常用模型**？
2. 概念类图、顺序图、状态图分别描述什么**视角**？
3. LLM 能自动生成 UML 图吗？**质量如何评估**？

PART 1 · 需求分析基础

为什么要需求分析 [SWEBOK 3.1]



需求分析是从**获取结果**到**共同理解**的桥梁。
它的核心目标：消除歧义、建立模型、为设计奠基。

需求分析的任务 [SWEBOK 3.1] [Bin2009]

任务一：建立分析模型

- 对需求获取阶段收集到的信息进行**整理和提炼**
- 建立分析模型，达成开发者和用户对需求信息的**共同理解**

任务二：创建解决方案

- 依据共同的理解，发挥**创造性**
- 创建软件系统的**解决方案**

分析模型是开发者与用户之间的"共同语言"

需求分析模型与建模 [SWEBOK 4.4]

"模型是对事物的抽象，帮助人们在创建一个事物之前可以有更好的理解" —— [Blaha2005]

建模的常用手段

手段	说明
抽象 (Abstraction)	忽略细节，聚焦本质特征和关键关系
分解 (Decomposition)	将复杂系统拆分为可理解的子部分

- 好的模型 = 合理的抽象层次 + 恰当的分解粒度
- 建模不是目的，**沟通和理解**才是目的

常见分析模型对照表 [SWEBOK 4.4]

方法	模型	描述
结构化方法	数据流图 (DFD)	从数据传递和加工角度, 描述功能处理过程
结构化方法	实体关系图 (ERD)	描述系统中的数据对象及其关系
面向对象方法	用例图 (Use Case Diagram)	描述用户与系统的交互, 说明系统边界和功能范围
面向对象方法	类图 (Class Diagram)	描述领域中重要概念及关系, 捕获系统静态结构
面向对象方法	交互图 / 顺序图 (Sequence Diagram)	描述一次交互的行为过程, 说明对象协作关系
面向对象方法	状态图 (State Diagram)	描述对象在整个生命期内的状态变化和行为过程

 本课程重点关注**面向对象方法**中的分析模型

面向对象分析的简单过程 [SWEBOK 4.4]

OO 需求分析模型包含

- 系统用例图、用例模型 / 用例描述
- 概念类图 / 领域模型
- 交互图（顺序图）
- 状态图
- 对象约束语言（OCL）

三个关键步骤

① 细化系统的对外交互
(用例图 → 用例描述)

② 明确用例中的协作对象
(概念类图)

③ 明确用例中的协作行为
(顺序图 / 状态图)



PART 2 · 概念类图

概念类图的定义 [SWEBOK 4.4]

- 描述应用领域中**重要的概念**以及概念之间的**关系**
- 捕获系统的**静态结构**
- 也称为"领域模型" (Domain Model)

概念类图 vs 设计类图

对比维度	概念类图（分析阶段）	设计类图（设计阶段）
关注点	问题域中的 概念和关系	软件内部的 类和接口
方法	通常 不包含 方法实现细节	包含方法签名和实现逻辑
来源	从 用例描述 和 领域知识 中提取	从概念类图 映射 而来

概念类图的建立过程 [SWEBOK 4.4]

四个步骤

步骤	活动	关键技巧
①	识别概念类	从用例描述中 提取名词 ，筛选有意义的概念
②	确定关联关系	识别概念之间的 "有关系" (association)
③	为概念类添加属性	识别描述概念类的 关键特征
④	确定关联的多重性	标注 1..* 、 0..1 等数量约束

 **名词提取法**是最常用的概念类识别方法——
用例描述中的名词往往就是潜在的概念类

名词分析法 —— Abbott 方法 [Abbott 1983]

五步走

步骤	活动	说明
①	收集用例描述文本	收集所有主要用例描述
②	标记名词和名词短语	逐句用 [] 标记所有名词
③	去除重复	汇总成列表，合并同义词
④	筛选	运用筛选准则排除不适合的名词
⑤	确认	与业务专家确认候选类列表

示例

"[顾客]选择'去结算'。[系统]显示[购物车]中的[商品清单]，包括每件[商品]的[名称]、[单价]、[数量]...[系统]生成[订单]"

→ 提取 26 个名词 → 经筛选保留 8 个概念类

候选类筛选五准则 [SWEBOK 4.4]

#	准则	排除示例
①	排除纯属性名词 — 只是某概念的简单属性值	"名称""单价""数量"→ 商品的属性
②	排除实现概念 — 分析阶段不含技术概念	"系统""结算流程"→ 不是问题域概念
③	排除外部系统 — 已作为参与者存在	"支付系统"→ 但"支付记录"可保留
④	排除含糊/重复名词	"商品清单"与"商品"重复
⑤	状态与行为测试 — 概念类应有属性且需被系统记录	"支付请求"→ 临时消息，不持久化

Peter Coad 概念类分类框架 —— 查漏检查表

分类	说明	在线书店示例
有形物体	可触摸的物理对象	Book（图书）
角色	人在系统中的角色	Customer、Administrator
事件/交易	特定时间发生的事，需记录	Order、Payment
场所	物理或逻辑位置	Address
容器	包含其他对象的对象	ShoppingCart
抽象概念	无形但有业务意义的概念	Category
描述/规格说明	描述其他对象的信息	Book（作为规格说明）
明细行	交易中的逐项记录	OrderLineItem、CartItem

💡 每类至少检查一遍——如果某类空缺，考虑是否有遗漏

示例：从需求获取信息到分析模型 [SWEBOK 4.4]

需求获取信息

"每本书都有至少一个作者"

转换过程

问题域语言

"每本书都有
至少一个
作者"



分析模型 (ER图)



软件内部实体

Book表
Wrote表
Author表

- 名词 → 概念类 (书、作者)
- 动词 → 关联关系 (写作)
- 数量词 → 多重性 (至少一个 → 1..*)

候选类向概念类的转化 [SWEBOK 4.4]

候选类特征	处理方式
既有状态，又有行为	确定为概念类
有状态，无行为	作为其他概念类的属性
无状态，有行为	先审视需求是否遗漏；若确认无遗漏，剔除该候选类，将行为转交给有状态支持的其他概念类
无状态，无行为	完全剔除

核心判据：状态是对象存在的基础，行为需要状态的支撑

概念类筛选案例 [SWEBOK 4.4]

超市 POS 系统的候选类筛选

候选类	状态	行为	结论
会员	积分、等级	积分累计、折扣计算	概念类
销售	金额、时间、状态	创建、完成、取消	概念类
顾客	—	—	剔除（无直接关联）
收据	—	—	剔除（无状态、无行为）
商品标识	编码、名称	—	作为商品类的属性

识别关联 [SWEBOK 4.4]

关联发现的三个来源

1. **分析用例文本描述** — 发现概念类之间的协作，需要协作的类之间建立关联
2. **分析问题域关系** — 补充整体-部分关系和明显的语义联系
3. **精简** — 去除冗余关联和导出关联

对问题域关系的补充要**适可而止**，不要把关系搞得过度复杂化

识别重要属性 [SWEBOK 4.4]

- 属性往往是实现类协作时必要的信息——协作的**条件、输入、结果或过程记录**
- 通过分析用例描述并与用户交流，补充问题域信息来发现重要属性
- 单个用例场景中发现的属性可能不多，需要**合并所有局部概念类图**后才能全面体现

属性发现流程

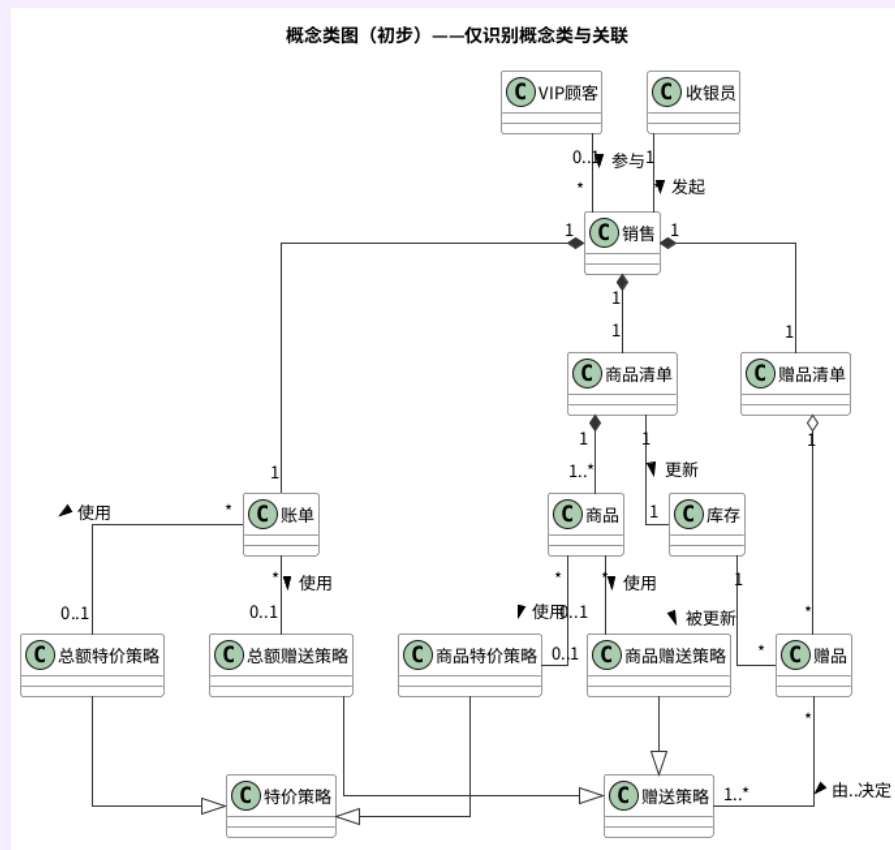
分析单个用例 —▶ 建立局部概念类图 —▶ 合并所有局部图 —▶ 完整属性集合



概念类图示例（初步）[SWEBOK 4.4]

仅识别概念类与关联

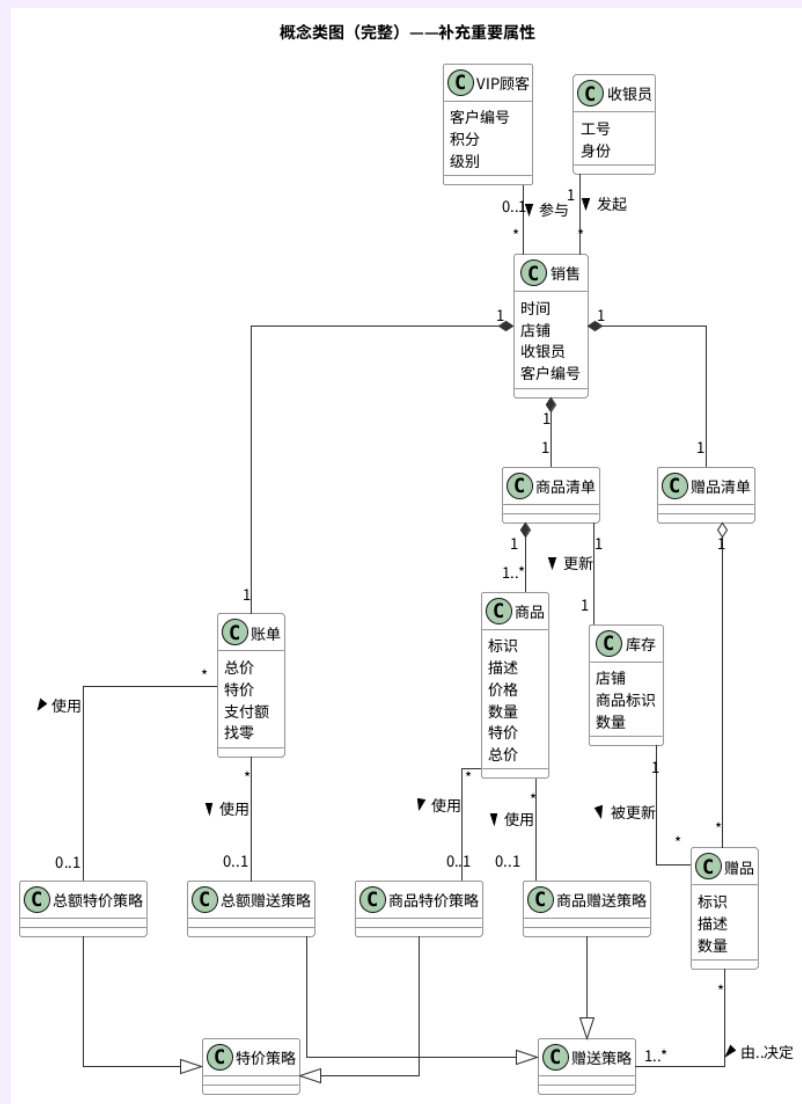
- 仅列出概念类名称和关联线
- 不含属性（属性在"完整版"补充）
- 关联带多重性标注（1、*、0..1）
- VIP顾客、收银员 → 与销售关联
- 销售 → 组合商品清单、账单
- 商品 → 关联库存、策略



概念类图示例（完整）[SWEBOK 4.4]

补充重要属性

- 销售：时间、店铺、收银员、客户编号
- 商品：标识、描述、价格、数量、特价、总价
- 账单：总价、特价、支付额、找零
- VIP顾客：客户编号、积分、级别
- 库存：店铺、商品标识、数量
- 收银员：工号、身份
- 赠品：标识、描述、数量
- 赠送策略 ← 商品赠送 / 总额赠送
- 特价策略 ← 商品特价 / 总额特价



案例对照 —— 在线书店概念类图 [SWEBOK 4.4]

概念类	关键属性	Coad 分类
Customer	用户名、邮箱、手机号	角色
Book	书名、作者、ISBN、价格	有形物体 / 规格说明
ShoppingCart	总金额	容器
CartItem	数量、小计	明细行
Order	订单编号、下单日期、总金额、状态	交易
OrderLineItem	数量、单价、小计	明细行
Address	收件人、手机号、省市区	场所
Payment	支付金额、方式、状态、流水号	交易
Category	类别名称、描述	抽象概念
Review	评分、内容、日期	事件
Inventory	库存数量、预警阈值	描述

在线书店 —— 关键关联关系 [SWEBOK 4.4]

关联	多重性	说明
Customer — ShoppingCart	1 : 1	每人一个购物车
ShoppingCart — CartItem	1 : *	购物车含多项
Customer — Order	1 : *	一人可下多单
Order — OrderLineItem	1 : 1..*	订单至少含一项
OrderLineItem — Book	* : 1	明细行引用图书
Order — Address	* : 1	多单可发同一地址
Order — Payment	1 : 1	一单一付
Book — Category	* : *	图书多分类
Book — Review	1 : *	图书有多条评价
Book — Inventory	1 : 1	一书一库存记录

概念类图建模检查清单

#	检查项
✓	所有用例描述中的 关键名词 都已分析？
✓	每个概念类都通过了 状态与行为测试 ？
✓	没有纯属性概念被误建模为独立的类？
✓	没有技术实现概念混入概念类图？
✓	所有关联的 多重性 都经过业务规则验证？
✓	关联类 的使用是否恰当？（如 OrderLineItem）
✓	概念类图能否支持 所有用例场景 ？（用例走查）
✓	概念类图是否已请 领域专家 审阅？

PART 3 · 顺序图

顺序图的概念 [SWEBOK 4.4]

- 描述系统中一次交互的行为过程
- 展示参与者、系统对象之间的消息传递时间顺序
- 从用例描述的**正常流程**映射而来

顺序图的核心元素

元素	含义
参与者 (Actor)	与系统交互的外部角色
生命线 (Lifeline)	对象在交互过程中的存在时间
消息 (Message)	对象之间传递的请求或响应
激活框 (Activation)	对象正在执行操作的时间段

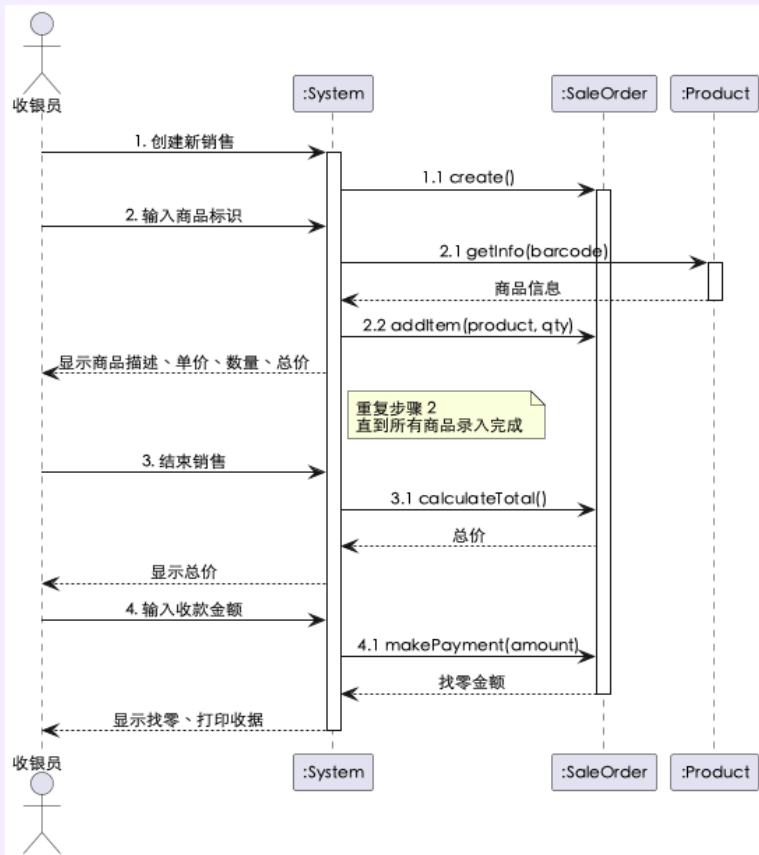
从用例描述到顺序图 [SWEBOK 4.4]

映射规则

用例描述中的内容	顺序图中的对应
参与者的 动作	从参与者到系统的 消息
系统的 响应	从系统到参与者的 返回消息
每一步 交互	一条 消息箭头
交互的 时间顺序	从上到下的 消息排列

用例描述中的每一步交互 → 顺序图中的一条消息。
正常流程先画，扩展流程用 `alt` / `opt` 片段补充。

顺序图示例：销售处理 创建与录入 [SWEBOK 4.4]



参与者与对象

- 收银员 (Actor) ↔ :System ↔ :SaleOrder ↔ :Product

关键消息

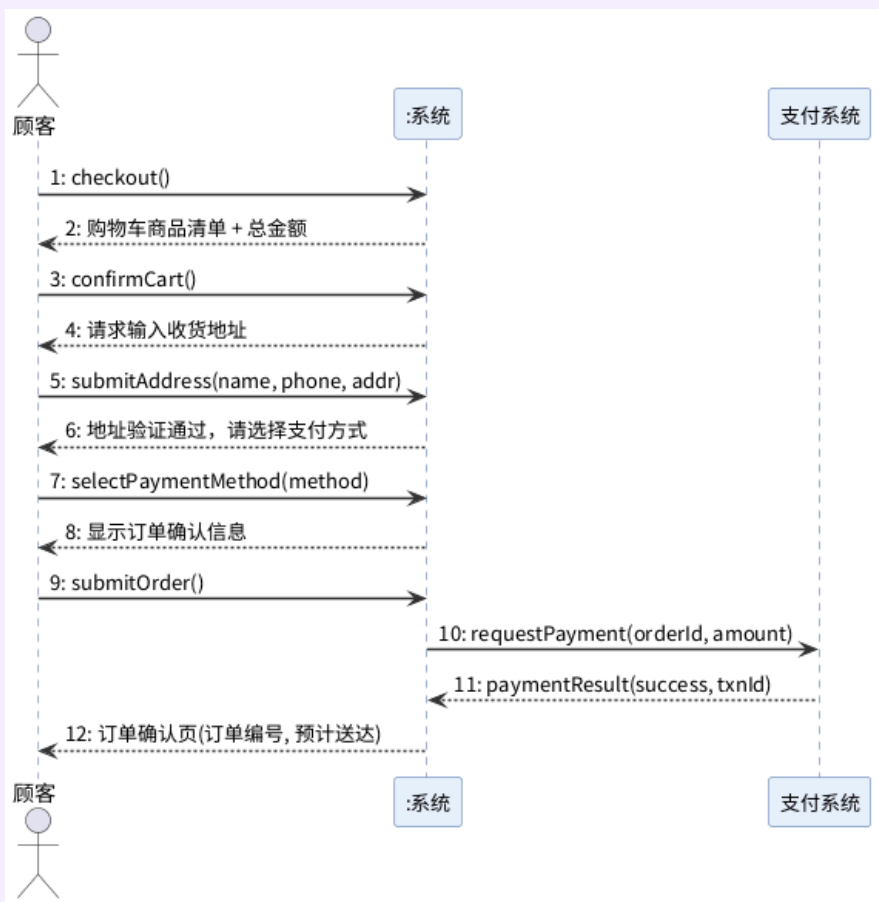
- 1.1 create() — 创建新销售
- 2.1 getInfo(barcode) — 查询商品信息
- 2.2 addItem(product, qty) — 添加商品
- 3.1 calculateTotal() — 计算总价
- 4.1 makePayment(amount) — 支付

交互片段

- 黄色注释框 = loop 片段：步骤 2 重复直到所有商品录入完成

顺序图从用例描述的正常流程映射而来，每一步交互对应一条消息箭头

案例 —— "购买图书"系统顺序图 [SWEBOK 4.4]



三方参与者

顾客 ↔ 系统 ↔ 支付系统

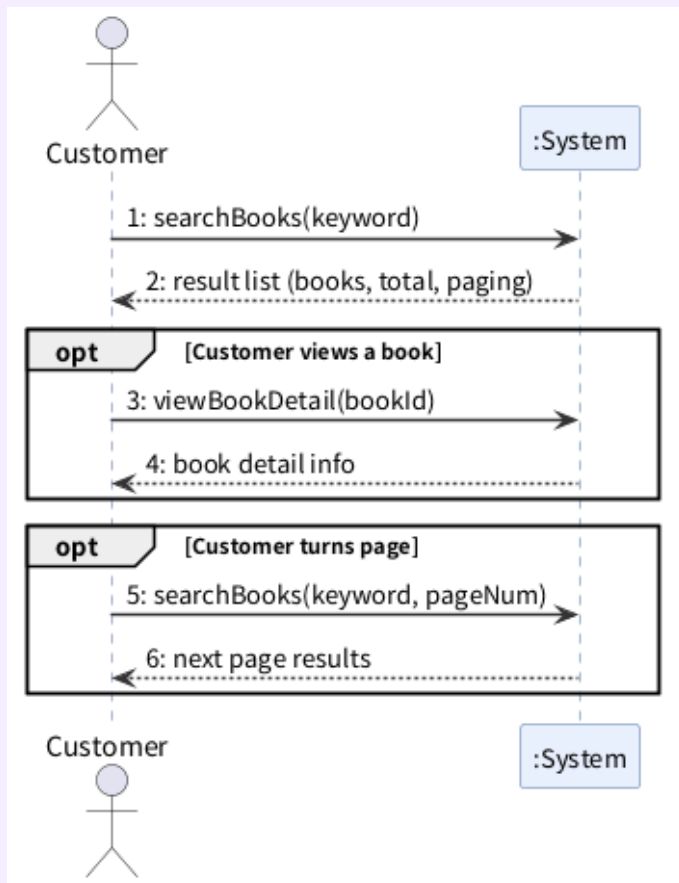
系统操作

- `checkout()` — 请求结算
- `confirmCart()` — 确认购物车
- `submitAddress(...)` — 提交地址
- `selectPaymentMethod()` — 选支付方式
- `submitOrder()` — 提交订单

外部系统交互

- `requestPayment()` → 支付系统
- `paymentResult()` ← 支付系统

案例 —— "搜索图书"系统顺序图 [SWEBOK 4.4]



系统操作

- `searchBooks(keyword)` — 基本搜索
- `viewBookDetail(bookId)` — 查看详情
- `searchBooks(keyword, pageNum)` — 翻页（方法重载）

交互片段

片段	含义	本图用法
opt	可选（0 或 1 次）	查看详情、翻页
alt	互斥分支	本图未用
loop	循环（0 到多次）	本图未用

Java 映射

```
// 方法重载实现翻页
SearchResult searchBooks(String kw)
SearchResult searchBooks(String kw, int page)
BookDetail viewBookDetail(String bookId)
```

系统操作契约（Operation Contract） [Larman 2004]

submitOrder() 的操作契约

字段	内容
操作名称	submitOrder()
交叉引用	UC-07"购买图书"步骤 9
前置条件	顾客已登录；存在"待确认"订单；包含 ≥ 1 明细行；地址和支付方式已填
后置条件	创建 Payment 实例并关联 Order；Order 状态→"待支付"；支付请求已发送；若支付成功：状态→"已支付"、库存扣减；若失败：状态不变

后置条件描述的是**什么改变了**（创建对象、修改属性、建立关联），不是怎么做

对象约束语言 OCL —— 让约束精确化 [OMG UML 2.5]

为什么需要 OCL?

UML 图能表达结构，但无法精确表达约束规则：

约束	UML 图的局限	OCL 的表达
订单总金额必须 > 0	类图画不出	<code>inv: self.totalAmount > 0</code>
购物车每件商品 1-99 件	多重性标不了	<code>inv: self.items->forAll(i i.qty <= 99)</code>
已发货不可取消	状态图用自然语言，有歧义	<code>pre: self.status <> #SHIPPED</code>

OCL 核心特点

- **声明式**：只说"是什么"，不说"怎么做"
- **无副作用**：不修改模型状态，只做断言和查询
- **依附于 UML**：附加在类图、状态图等模型元素上

OCL 核心语法 [OMG UML 2.5]

1. 不变量 (Invariant) —— 对象始终满足的条件

```
context Order
inv: self.lineItems->size() >= 1      -- 订单至少含一个明细行
inv: self.totalAmount > 0             -- 总金额为正
```

2. 前置/后置条件 —— 操作契约的形式化

```
context Order::pay()
pre:  self.status = OrderStatus::PENDING_PAYMENT
post: self.status = OrderStatus::PAID
      and self.inventory.quantity = self.inventory.quantity@pre - self.quantity
```

`@pre` 引用操作执行前的值——这是 OCL 独特且强大的机制

3. 导航 —— 沿关联路径访问对象

```
context OrderLineItem
inv: self.order.customer.isActive = true
-- 导航路径: OrderLineItem → Order → Customer
```

OCLE 集合操作速查 [OMG UML 2.5]

操作	含义	示例
->size()	集合大小	self.items->size() >= 1
->forAll(e expr)	所有元素满足	items->forAll(i i.qty > 0)
->exists(e expr)	存在元素满足	items->exists(i i.price > 100)
->select(e expr)	过滤	orders->select(o o.status = #PAID)
->collect(e expr)	映射	items->collect(i i.subtotal)
->sum()	求和	->collect(i i.subtotal)->sum()
->isUnique(e expr)	唯一性	books->isUnique(b b.isbn)

OCLE 与本课程已有内容的对应

课程内容	OCLE 对应
概念类图多重性 1..*	inv: self.lineItems->size() >= 1
状态图守卫条件	pre: self.status = #PENDING_PAYMENT
操作契约（前/后置条件）	pre: / post: + @pre
业务规则 BR1（最大99件）	inv: self.quantity <= 99

用例模型 → Java 代码的映射 [SWEBOK 4.4]

用例模型中的元素	Java 代码中的对应
参与者的动作	HTTP 请求、用户事件
系统的响应	Controller 方法的返回值
主成功场景	Service 层的 正常业务逻辑
替代场景	异常处理 、条件分支
前置条件	方法开头的 参数校验和状态检查
后置条件	方法执行后的 系统状态变化
业务规则	Service 层的 业务验证逻辑
一个完整用例	一个 Controller 方法 + 多个 Service 方法协作

用例描述中的每一步 → 将来代码的设计依据

PART 4 · 状态图

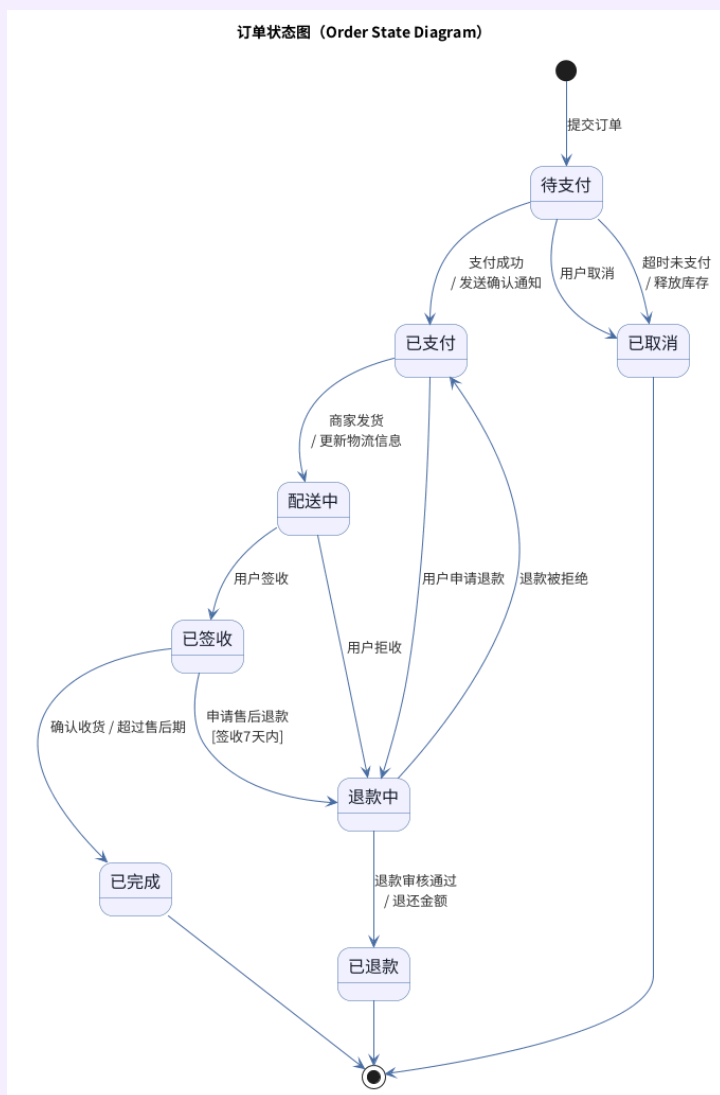
状态图概念 [SWEBOK 4.4]

- 描述系统、用例或对象在**整个生命期内的状态变化**
- 适用场景：订单状态流转、设备状态、用户账户状态等
- 也称为"状态机图" (State Machine Diagram)

基本元素

元素	符号	说明
状态 (State)	圆角矩形	对象在某一时刻的条件或情况
转换 (Transition)	箭头	从一个状态到另一个状态的变迁
事件 (Event)	转换上的标签	触发状态转换的外部刺激
动作 (Action)	/ action	状态转换时执行的操作
初始状态	实心圆	状态机的起点
终止状态	圆中圆	状态机的终点

状态图示例：订单系统 [SWEBOK 4.4]



状态图示例：订单系统 — 解读 [SWEBOK 4.4]

状态转换路径

状态	触发事件	目标状态
待支付	支付成功 / 发送确认通知	已支付
待支付	用户取消 或 超时未支付 / 释放库存	已取消
已支付	商家发货 / 更新物流信息	配送中
配送中	用户签收	已签收
配送中	用户拒收 或 申请退款	退款中
已签收	确认收货 / 超过售后期	已完成
已签收	申请售后退款 [签收7天内]	退款中
退款中	退款审核通过 / 退还金额	已退款

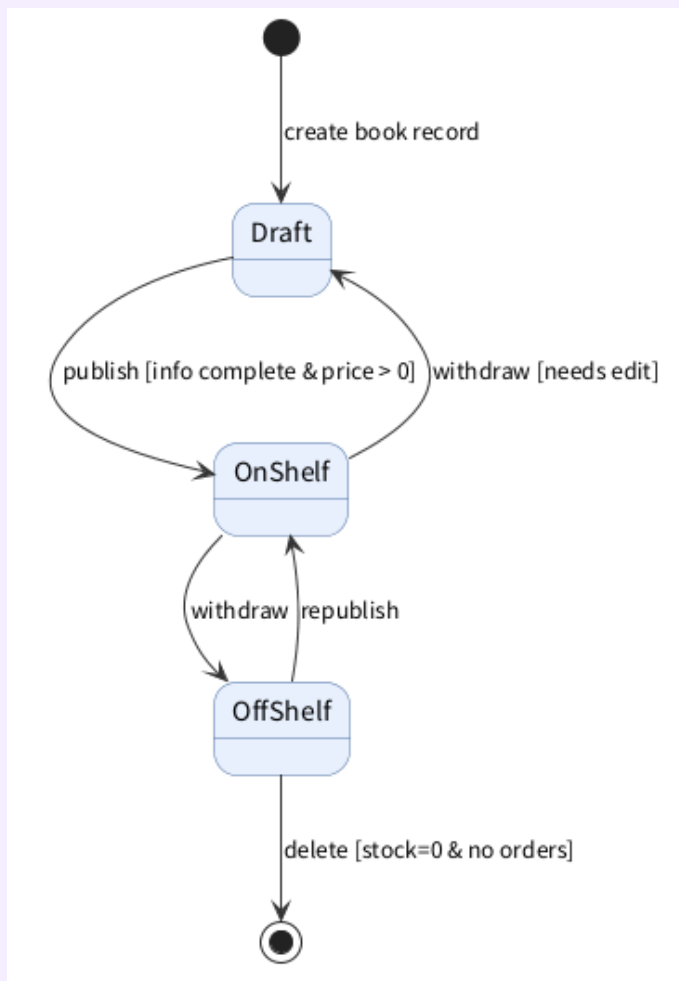
状态图的适用场景 [SWEBOK 4.4]

状态图关注的是**单个对象**在不同事件驱动下的行为变化，适合描述生命周期复杂的实体（订单、设备、会话等）。

典型适用对象

对象类型	状态示例
订单	待支付 → 已支付 → 已发货 → 已完成
设备	空闲 → 运行中 → 故障 → 维修中
用户会话	未登录 → 已登录 → 超时 → 已注销

案例 —— 图书上架状态图 [SWEBOK 4.4]



三状态生命周期

状态	说明
Draft	管理员创建后的初始状态
OnShelf	对顾客可见，可被购买
OffShelf	对顾客不可见，可重新上架或删除

守卫条件

- 上架: `[info complete & price > 0]`
- 删除: `[stock = 0 & no orders]`

Java 枚举

```
public enum BookStatus {
    DRAFT, ON_SHELF, OFF_SHELF
}
```

与订单状态图对比：图书状态更简单，但同样需要守卫条件防止非法转换

状态图 → Java 代码的映射 [SWEBOK 4.4]

映射规则

状态图元素	Java 对应
状态	枚举值
转换	方法
守卫条件	if 检查
非法转换	抛出异常
动作	方法内逻辑

状态图中的每个转换 → 一个方法；每个守卫条件 → 方法开头的 if 检查

代码示例

```
public enum OrderStatus {
    PENDING_PAYMENT, PAID, SHIPPED,
    COMPLETED, CANCELLED, REFUNDING
}

// 待支付 → 已支付
public void pay() {
    if (status != PENDING_PAYMENT)
        throw new IllegalStateException();
    status = PAID;
    deductInventory();      // 动作：扣减库存
}

// 已支付 → 已发货
public void ship(String trackingNo) {
    if (status != PAID)
        throw new IllegalStateException();
    status = SHIPPED;
}

// 多条 → 已取消（守卫条件不同）
public void cancel() {
    if (status == PENDING_PAYMENT) {
        status = CANCELLED;
        releaseInventory();    // 释放库存
    } else if (status == PAID) {
        status = CANCELLED;
        releaseInventory();
        initiateRefund();      // 发起退款
    } else throw new IllegalStateException();
}
```

状态图指导测试用例编写 [SWEBOK 4.4]

为每条转换路径编写测试

测试用例	初始状态	操作	预期结果
正常支付	PENDING_PAYMENT	<code>pay()</code>	状态 → PAID
已发货不可支付	SHIPPED	<code>pay()</code>	抛出 <code>IllegalStateException</code>
已支付可取消	PAID	<code>cancel()</code>	状态 → CANCELLED, 库存退还
已完成不可取消	COMPLETED	<code>cancel()</code>	抛出 <code>IllegalStateException</code>

状态图中的每条转换路径 + 每种非法转换 = 完整的测试覆盖

PART 5 · AI 辅助 UML 图生成

LLM + UML 工具链

工具	特点	LLM 配合方式
PlantUML	覆盖全部 UML 图类型，语法严谨	LLM 生成 PlantUML 代码 → 渲染引擎出图
Mermaid	JavaScript 原生，GitHub / Markdown 内嵌	LLM 生成 Mermaid 代码 → 浏览器直接渲染
Claude Artifacts	直接生成交互式可视化图表	对话式生成，即时预览

典型 workflow

需求描述（自然语言） —▶ LLM 生成代码 —▶ PlantUML/Mermaid 渲染 —▶ 人工审查修正

来源：dev.to/mike-vincent, thenewstack.io

实操演示：让 LLM 生成 PlantUML 用例图

Prompt 示例：

你是一名 UML 建模专家。请根据以下业务描述，生成连锁超市销售系统的 PlantUML 用例图代码。

系统参与者包括：总经理、客户经理、收银员、管理员、VIP顾客。

主要功能包括：

- 收银员进行销售处理和退货处理
- 客户经理管理客户信息和VIP卡
- 总经理查看销售报表
- 管理员管理商品和库存
- VIP顾客享受积分兑换服务

输出纯 PlantUML 代码，不要解释。

 **关键：**明确列出参与者和功能，LLM 才能生成准确的用例图

实操演示：让 LLM 生成 Mermaid 顺序图

Prompt 示例：

请根据以下用例描述，生成销售处理用例的 Mermaid 顺序图代码。

参与者：收银员(Cashier)、系统(System)

正常流程：

1. 收银员开始新的销售
2. 收银员输入商品条码
3. 系统显示商品信息和单价
4. 收银员输入商品数量
5. 系统记录销售明细并更新小计
6. 重复步骤 2-5 直到所有商品录入完毕
7. 收银员结束销售
8. 系统计算总价
9. 收银员输入收款金额
10. 系统计算找零并打印小票

请使用 Mermaid sequenceDiagram 语法输出。

LLM 生成各类 UML 图的质量评测

图类型	LLM 表现	关键问题
用例图	较好	<code>include</code> / <code>extend</code> 关系容易误用
类图	中等	关联关系和多重性判断不准确
顺序图	中等偏下	消息顺序、激活框处理差
状态图	较弱	状态转换条件容易遗漏

关键发现

- 图越大越复杂，**质量越差**——建议拆解为小的、聚焦的组件
- 不同 LLM 表现有差异，需实际测试选择
- 所有 LLM 生成的 UML 都需要人工审查

来源：arxiv.org/html/2603.09100, arxiv.org/html/2404.06371v2

MermaidSeqBench — 顺序图生成基准测试

IBM Research 发布 · NeurIPS 2025

- 132 个人工验证样本，覆盖多种业务场景

6 个评分维度

#	维度	评估内容
1	Syntax（语法正确性）	代码能否被解析器正确解析
2	Mermaid Only（语法规范）	是否符合 Mermaid 顺序图语法规范
3	Logic（逻辑正确性）	消息顺序和参与者交互是否合理
4	Completeness（完整性）	是否覆盖了所有关键交互步骤
5	Activation Handling（激活框处理）	激活/停用时机是否正确
6	Error & Status Tracking（错误追踪）	异常流程和状态码是否完整

结果：各模型在**激活框处理**和**错误追踪**方面存在显著能力差距

来源：arxiv.org/abs/2511.14967

课堂实操（20 分钟）

任务：以电梯系统为例

步骤	任务内容	输出物
①	让 LLM 生成 PlantUML 用例图	用例图代码 + 渲染图
②	让 LLM 生成 Mermaid 顺序图（乘电梯用例）	顺序图代码 + 渲染图
③	让 LLM 生成 PlantUML 概念类图	类图代码 + 渲染图
④	用 MermaidSeqBench 的 6 维度为顺序图打分	评分表
⑤	找出 AI 输出中的 错误 ，手动修正	修正后的代码

记录与思考：哪类图 AI 生成质量最高？哪类最差？为什么？

AI 建模的最佳实践

五条核心原则

#	原则	说明
①	拆解再生成	将大系统拆解为小的、聚焦的组件再让 AI 生成
②	提供明确上下文	给出参与者列表、用例描述、业务规则
③	指定输出格式	明确要求 PlantUML 或 Mermaid 语法
④	必须人工审查	重点检查关系、多重性、消息顺序、状态转换
⑤	Few-shot 示范	提供一个正确示例，再让 AI 生成其他

AI 生成的是初稿，不是终稿。

工程师的价值在于审查、修正和决策。

课堂总结

今日要点回顾

主题	关键结论
需求分析	通过 建模 达成开发者与用户的共同理解
OO 分析核心产出	用例图 → 概念类图 → 顺序图 → 状态图
AI 辅助建模	LLM + PlantUML / Mermaid 可 快速生成 UML 初稿
AI 的弱点	关系判断、激活框处理、状态转换条件
人工审查	不可省略 ——AI 是助手，不是替代者

回答核心问题

1. 需求分析通过建立分析模型，消除歧义，达成**共同理解**
2. 类图看**静态结构**，顺序图看**交互行为**，状态图看**生命周期**
3. LLM 能生成 UML，但质量参差——需用**多维度标准**评估

课后作业

以本组项目为背景，完成以下任务

任务	要求
① 手动建模	选择 2 个核心用例，手动建立概念类图和顺序图
② AI 生成	用 LLM 生成同样的图（PlantUML 代码），渲染后保存
③ 质量评分	用 6 维度评分表为 AI 生成的顺序图打分
④ 对比分析	对比手绘版与 AI 版，分析差异和改进方向

提交物

- 手绘版图（拍照或工具绘制）
- AI 版图（PlantUML/Mermaid 代码 + 渲染图）
- 6 维度评分表
- 对比分析报告（不少于 300 字）

本节课结束

"模型是思考的工具。AI 生成的是初稿，工程师的审查赋予它正确性。"